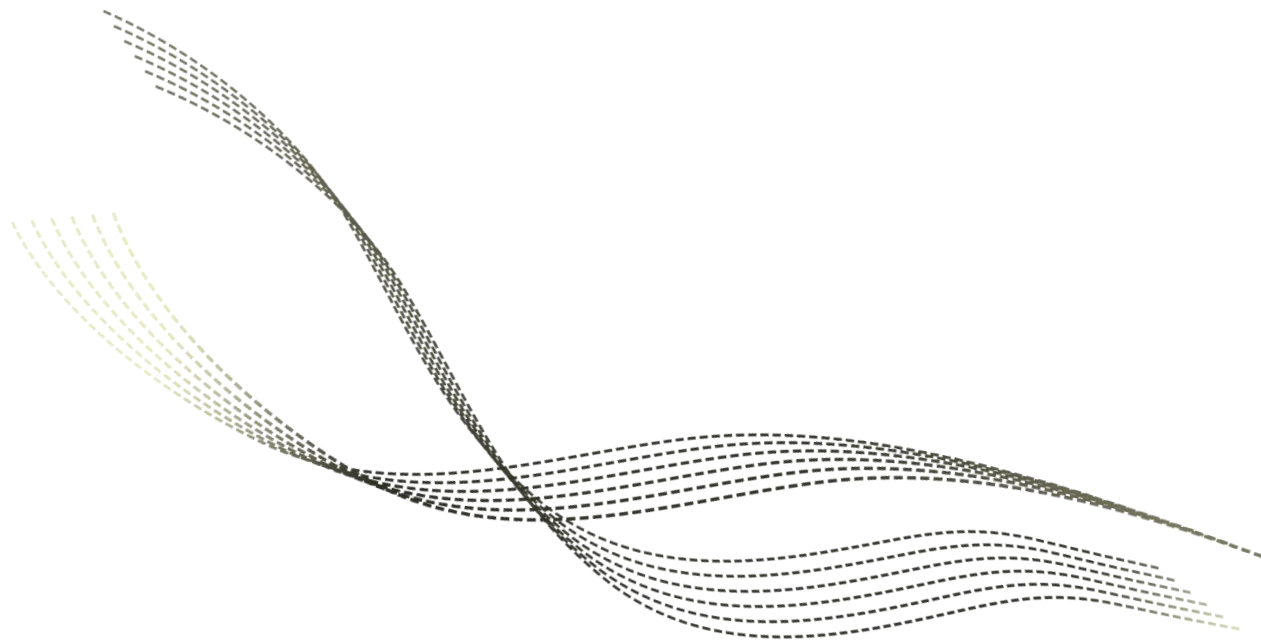


超大规模集成电路设计导论

Very Large Scale Integration Circuit (VLSI) Design

集成电路版图规划方法

沈明华



目录

CONTENTS

01

布图规划

02

贪心算法

03

搜索算法

04

线性规划

PART 01

布图规划

■ 什么是布图规划 (Floorplanning)

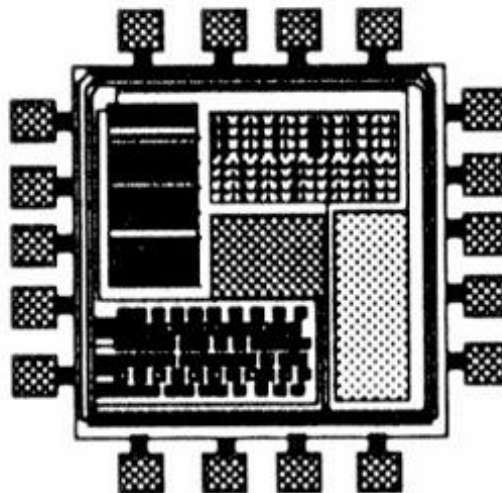
□ 房型图

- 以房间简图为例，排布这些房间的位置、设计房间的大小，使得房间总面积不超过宅基地面积，同时让房型尽可能舒适



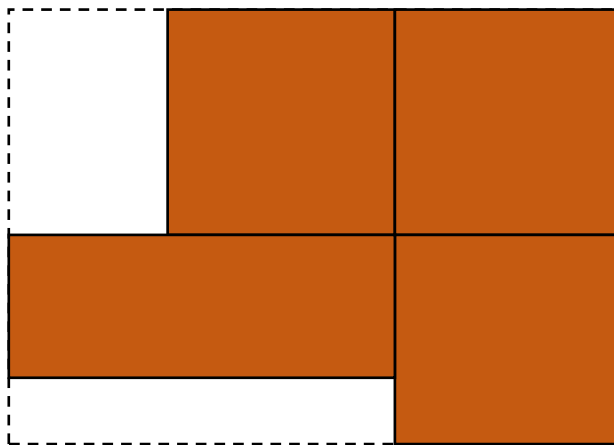
■ 布图规划

- 上一步我们把芯片划分成多个模块
 - 但距离芯片成品还有许多步骤要走
- 经过划分后的几个模块，我们需要考虑
 - 哪些模块放在一起
 - 每个模块的形状
 - 模块大概放哪
- 先走哪一步？



■ 布图规划

- 在不了解模块的形状和I/O引脚位置的情况下，如何将这些模块放置在一起？
- 如果我们先设计这些模块的形状，可能会导致它们无法紧密排列



■ 布图规划

- **布图规划是在芯片设计初期阶段规划模块的位置和形状（这是相对的）**
- 目标是优化电路性能，包括：
 - 芯片面积
 - 总线长
 - 关键路径时延
 - 可布线性
 - 其他，例如噪声、散热等

■ 布图规划(Floorplan)与布局(Placement)区别

□ 两者都决定了模块的位置，都以优化电路性能为目标

– 布图规划：

- 细节部分尚未确定
- 软模块的形状、IO引脚位置、模块的位置都不确定

– 布局：

- 细节部分是确定的
- 只有部分模块的位置与方向是不确定的

– 定义

- 软模块：形状可变的模块，可以自行按需求修改模块形状
- 硬模块：形状不可变的模块

■ 布图规划问题

□ 问题定义

– 输入:

- n 个模块, 面积分别为 $A_1, A_2, \dots, A_n$
- r_i 和 s_i 是关于模块 B_i 的长宽比的上下界

– 输出:

- 每个模块的坐标 (x_i, y_i) 、宽度 w_i 和高度 h_i , 满足 $h_i w_i = A_i$ 且 $r_i \leq h_i/w_i \leq s_i$

➤ **模块间没有重叠**

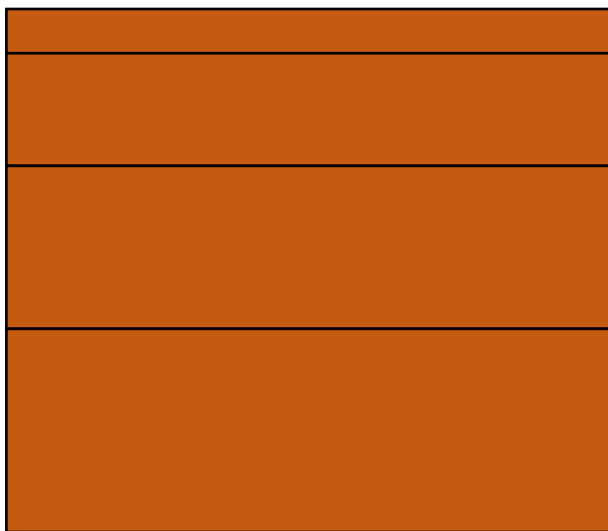
– 目标:

- 优化芯片的性能

■ 长宽比的界限

□ 如果没有长宽比的限制，我们能否将所有模块紧密排列？

– 当然可以

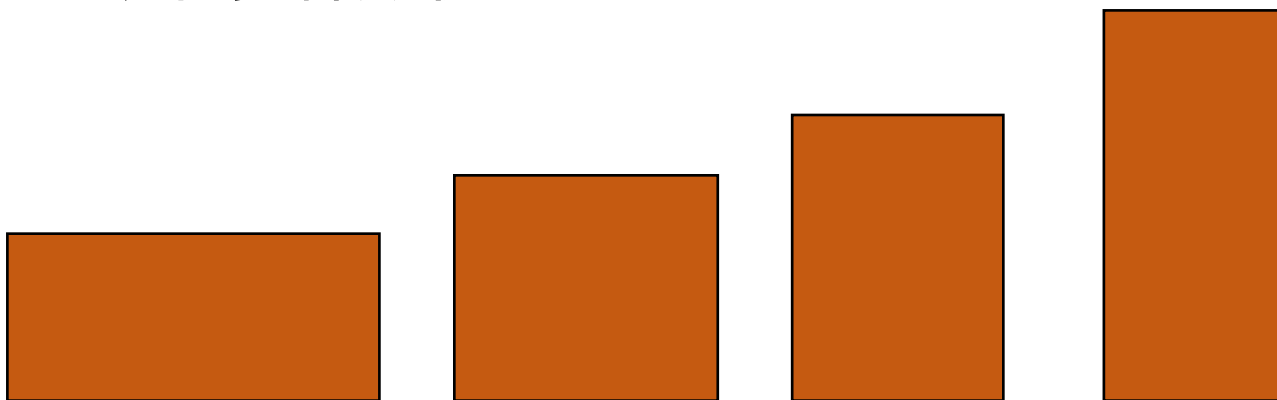


□ 但我们不希望将模块布局成长条状，因此我们要求每个模块的长宽比

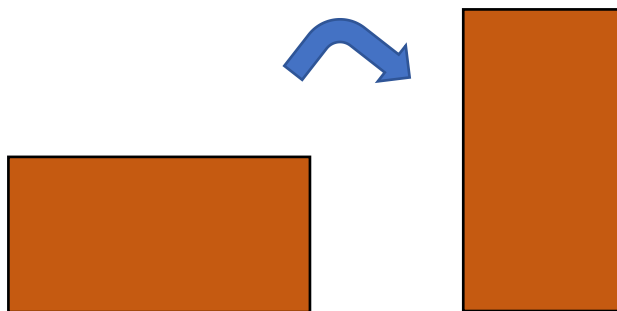
都满足 $r_i \leq \frac{h_i}{w_i} \leq s_i$ 。

■ 长宽比的界限

□ 软模块：可以有多种形状：



□ 硬模块：形状不可变，但可以改变其朝向：



■ 目标函数(Objective Function)

□ 常用的目标函数是面积和线长的加权和

$$- cost = \alpha A + \beta L$$

□ 其中 A 是总面积, L 是总布线长度, α 和 β 是常数

– 主要目标是

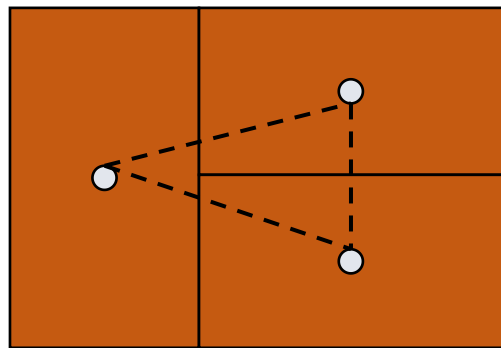
➤ 最小化芯片面积

➤ 最小化**总布线长度**

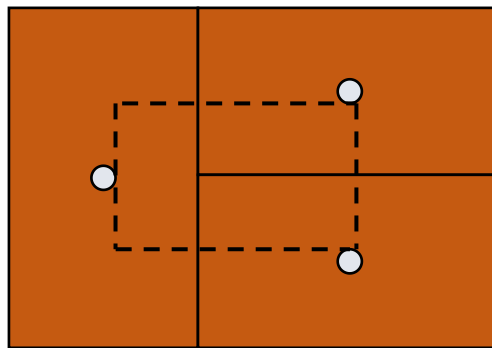
■ 线长估计(Wirelength Estimation)

- 在布线设计完成之前，不知道精确布线长度
- 在布图规划阶段，甚至连引脚的位置也尚未确定
- 所以我们使用一些**简单的线长估计算法**来**预估**当前结果好坏：
 - 中心到中心估算 (Center-to-center estimation)
 - 半周长估算 (Half-perimeter estimation)

中心到中心估算

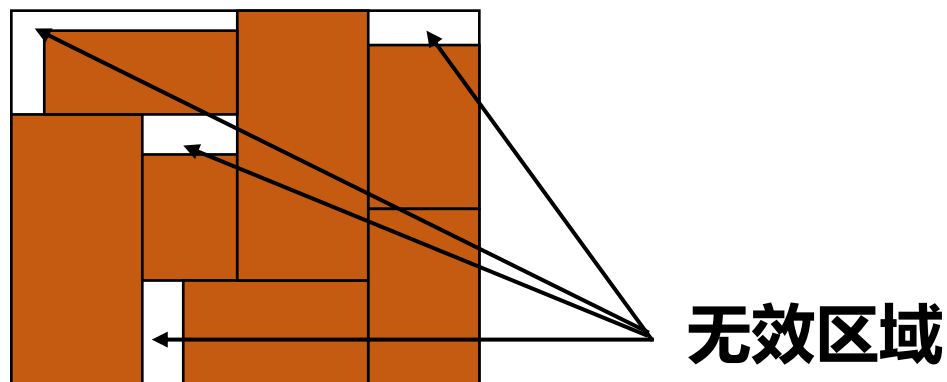


半周长估算



■ 无效区域(Dead space)

□ 无效区域是被浪费的空间：



- 布图总面积是布图规划评价指标之一，而电路模块占用的面积是固定的。
- 能影响最终布图面积只有模块间无法利用的细小空间碎片即无效区域。
- 布图规划过程要最小化无效区域的面积。

□ **最小化面积**等同于**最小化无效区域**

□ 无效区域百分比计算公式为，其中A是整个芯片的面积

$$(A - \sum_i A_i) / A \times 100\%$$

PART 02

贪心算法

■ 切片树

□ 问题建模——切片树^[1]

– 切片式布图规划(slicing floorplan):

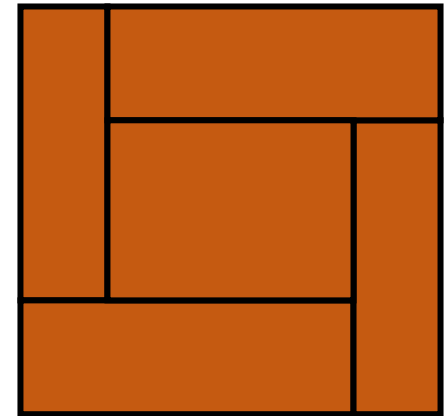
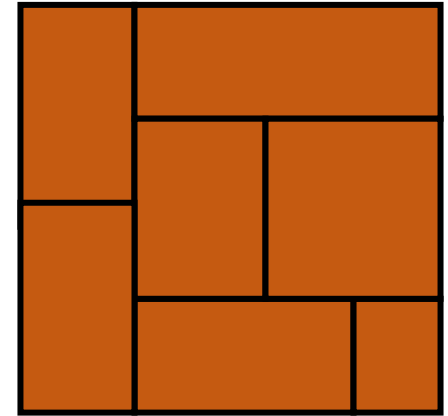
➤ 通过重复地水平或垂直分割（切片）矩形得到的布局

➤ **每一刀必须切到底**

– 非切片式布图规划(non-slicing floorplan):

➤ 无法仅通过重复分割得到的布局

– Otten (LSSS-82) ^[2]指出: **切片式布图规划更容易处理**



[1] D.F. Wong, C.L. Liu, "A new algorithm for floorplan design," DAC, 1986.

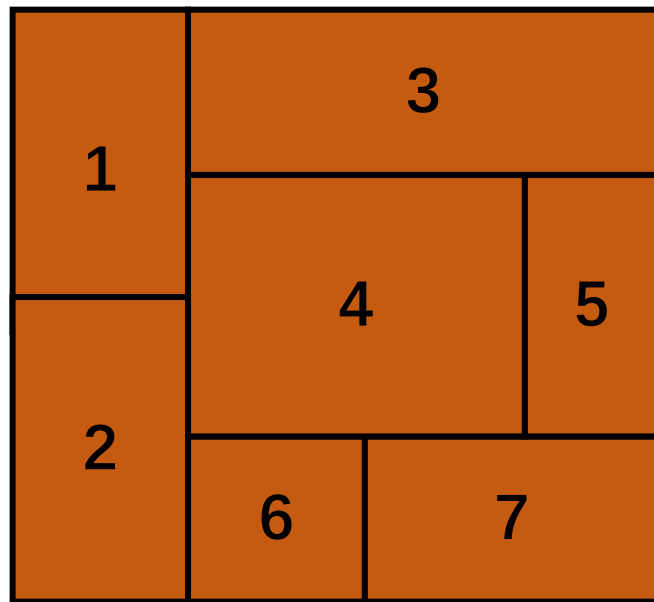
[2] Ralph H.J.M Otten, "Automatic floorplan design," DAC, 1982.

■ 切片树

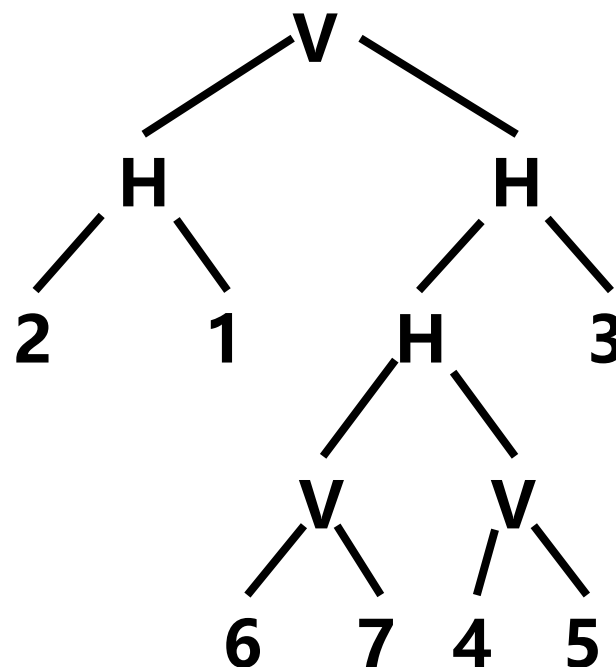
□ 问题建模

– 切片式布图规划可以用切片树表示

切片式布图规划



切片树

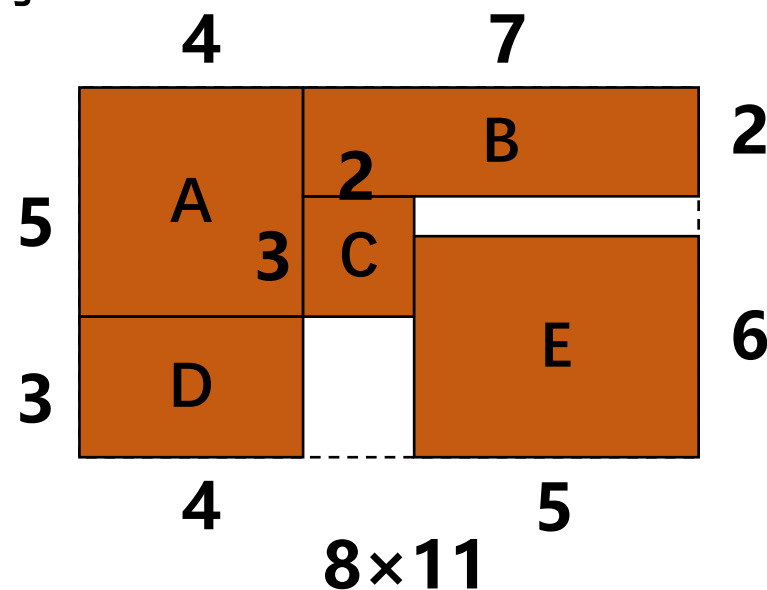
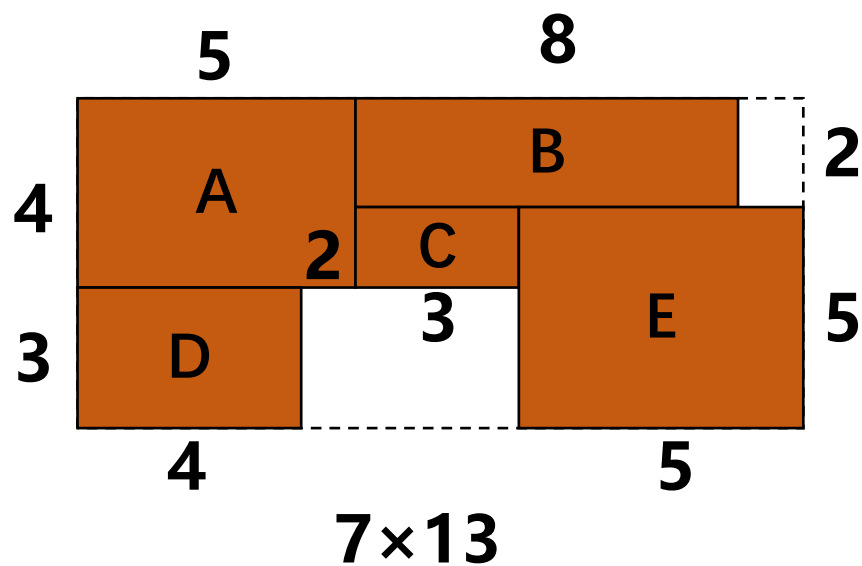


H表示水平切
V表示垂直切

■ Stockmeyer算法

□ 从切片树到布图规划

- 《Optimal Orientation of Cells in Slicing floorplan Designs》^[1]
 - 这是早于[Wong-Liu'86]^[2]的一篇论文，处理的是更简单的问题
- **问题：**给定**一组模块的形状**列表
 - 如何**放置**这些模块使得**占据的总面积**最小



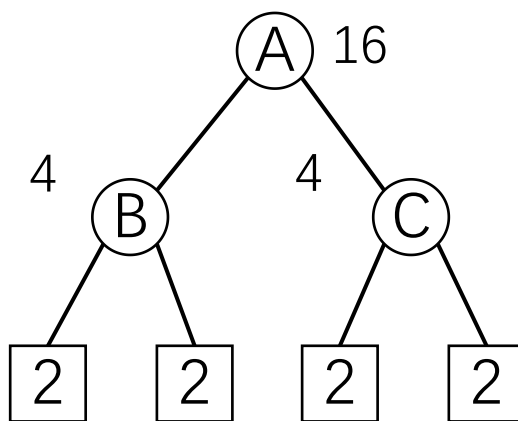
[1] Larry Stockmeyer, "Optimal orientations of cells in slicing floorplan designs," Information and Control, 1986.

[2] D.F. Wong, C.L. Liu, "A new algorithm for floorplan design," DAC, 1986.

■ Stockmeyer算法

□ 放置问题的难点在于，组合数量爆炸

- 底下**第一层叶子节点**表示**每个模块之间有2种可能的宽高**(即**横放**或**竖放**)
- B、C第二层节点就有4种
- 第三层的A节点就有 $4 \times 4 = 16$ 种
- 随着节点数量的增加，整个问题规模很快就膨胀到不可计算



■ Stockmeyer算法

□ 算法分为两个阶段

– 自下而上遍历树——动态规划

➤ 输入：**切片树**，**模块固定形状**

➤ 从最底层叶子节点开始，

如果是**水平**切割，则执行**水平**切割的**合并**；

如果是**垂直**切割，则执行**垂直**切割的**合并**，

两种合并的输入列表排序方法不同：前者按宽度降序排列，后者按高度降序排列。

合并后得到一个有序列表，排序方式与输入列表相同。

➤ 合并后保留所有可能的**局部最优解**，作为**下一次合并**的对象

➤ 当**遍历**到**根节点**时，**选择列表中构成最小面积的一项**

– 自上而下遍历树

➤ 遍历切片树并且确定模块的布局结果

■ Stockmeyer算法

算法分为两个阶段

– 自下而上遍历树——动态规划

- 输入：切片树，模块固定形状
- 从最底层叶子节点开始，
如果是水平切割，则执行水平切割的合并；
如果是垂直切割，则执行垂直切割的合并，
两种合并的输入列表排序方法不同：
前者按宽度降序排列，后者按高度降序排列
合并后得到一个有序列表，排序方式与输入
- 合并后保留所有可能的局部最优解，作为T
- 当遍历到根节点时，选择列表中构成最小值

– 自上而下遍历树

- 遍历切片树并且确定模块的布局结果

算法 1 水平切割的合并

Input: 两个排序好的列表 v 和 v' ,

其中 $\forall i < j$, 有 $w_i \geq w_j$ 和 $h_i \leq h_j$

Output: 合并好的列表 u

```
1: 初始化  $i \leftarrow 1, j \leftarrow 1, k \leftarrow |v|, m \leftarrow |v'|$ 
2: while  $i \leq k$  且  $j \leq m$  do
3:   在列表  $u$  中加入新合并的节点  $(\max(w_i, w'_j), h_i + h'_j)$ 
4:   if  $w_i > w'_j$  then
5:      $i \leftarrow i + 1$ 
6:   else if  $w_i < w'_j$  then
7:      $j \leftarrow j + 1$ 
8:   else
9:      $i \leftarrow i + 1, j \leftarrow j + 1$ 
10:  end if
11: end while
```

■ Stockmeyer算法

算法分为两个阶段

– 自下而上遍历树——动态规划

- 输入：**切片树**，**模块固定形状**
- 从最底层叶子节点开始，
如果是**水平**切割，则执行**水平**切割的**合并**；
如果是**垂直**切割，则执行**垂直**切割的**合并**，
两种合并的输入列表排序方法不同：
前者按宽度降序排列，后者按高度降序排列
合并后得到一个有序列表，排序方式与输入
- 合并后保留所有可能的**局部最优解**，作为**T**
- 当**遍历**到**根节点**时，**选择列表中构成最小值**

– 自上而下遍历树

- 遍历切片树并且确定模块的布局结果

算法 2 垂直切割的合并

Input: 两个排序好的列表 v 和 v' ,

其中 $\forall i < j$, 有 $w_i \leq w_j$ 和 $h_i \geq h_j$

Output: 合并好的列表 u

```
1: 初始化  $i \leftarrow 1, j \leftarrow 1, k \leftarrow |v|, m \leftarrow |v'|$ 
2: while  $i \leq k$  且  $j \leq m$  do
3:   在列表 $u$ 中加入新合并的节点 $(w_i + w'_j, \max(h_i, h'_j))$ 
4:   if  $h_i > h'_j$  then
5:      $i \leftarrow i + 1$ 
6:   else if  $h_i < h'_j$  then
7:      $j \leftarrow j + 1$ 
8:   else
9:      $i \leftarrow i + 1, j \leftarrow j + 1$ 
10:  end if
11: end while
```

■ Stockmeyer算法

□ 水平合并和垂直合并的算法流程

算法 1 水平切割的合并

Input: 两个排序好的列表 v 和 v' ,

其中 $\forall i < j$, 有 $w_i \geq w_j$ 和 $h_i \leq h_j$

Output: 合并好的列表 u

```
1: 初始化 $i \leftarrow 1, j \leftarrow 1, k \leftarrow |v|, m \leftarrow |v'|$ 
2: while  $i \leq k$  且  $j \leq m$  do
3:   在列表 $u$ 中加入新合并的节点( $\max(w_i, w'_j), h_i + h'_j$ )
4:   if  $w_i > w'_j$  then
5:      $i \leftarrow i + 1$ 
6:   else if  $w_i < w'_j$  then
7:      $j \leftarrow j + 1$ 
8:   else
9:      $i \leftarrow i + 1, j \leftarrow j + 1$ 
10:  end if
11: end while
```

算法 2 垂直切割的合并

Input: 两个排序好的列表 v 和 v' ,

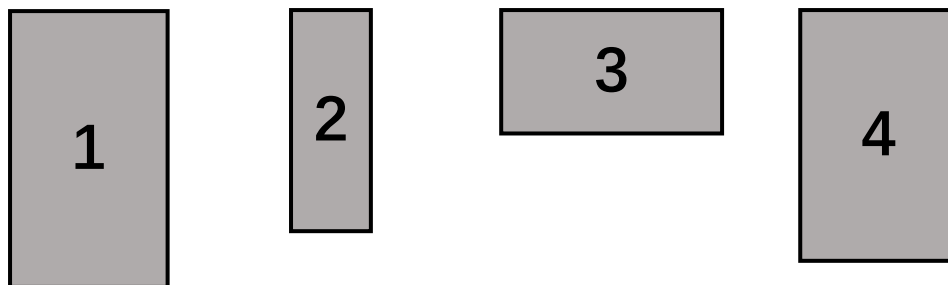
其中 $\forall i < j$, 有 $w_i \leq w_j$ 和 $h_i \geq h_j$

Output: 合并好的列表 u

```
1: 初始化 $i \leftarrow 1, j \leftarrow 1, k \leftarrow |v|, m \leftarrow |v'|$ 
2: while  $i \leq k$  且  $j \leq m$  do
3:   在列表 $u$ 中加入新合并的节点( $w_i + w'_j, \max(h_i, h'_j)$ )
4:   if  $h_i > h'_j$  then
5:      $i \leftarrow i + 1$ 
6:   else if  $h_i < h'_j$  then
7:      $j \leftarrow j + 1$ 
8:   else
9:      $i \leftarrow i + 1, j \leftarrow j + 1$ 
10:  end if
11: end while
```

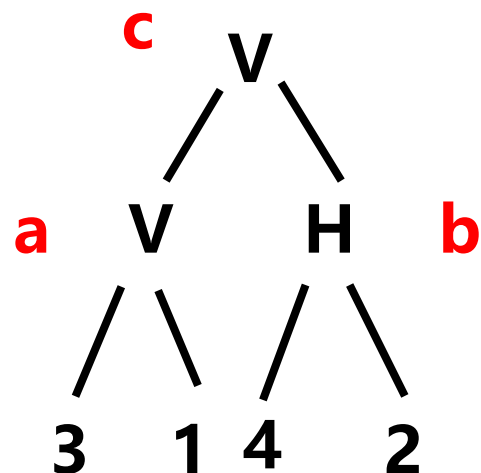
■ Stockmeyer算法

□ 输入:

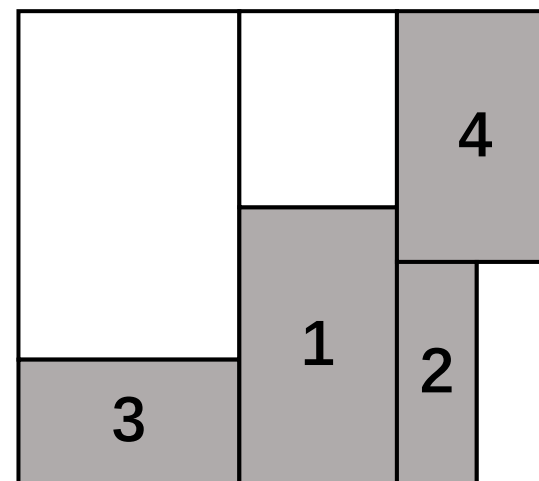


□ 模块的形状大小: $(2, 4)$, $(1, 3)$, $(3, 2)$, $(2, 4)$

□ 切片树: 已经固定了切法



得到
→

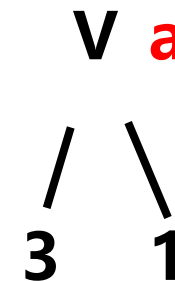


初始状态

■ Stockmeyer算法

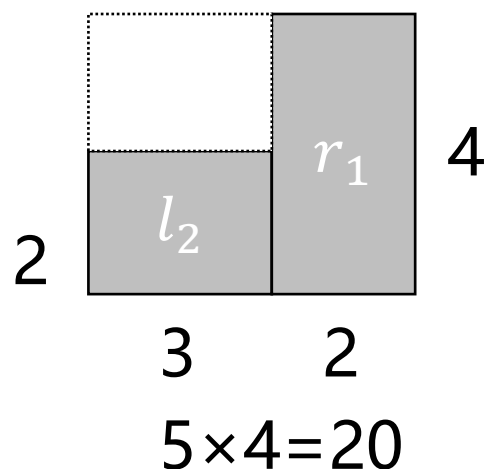
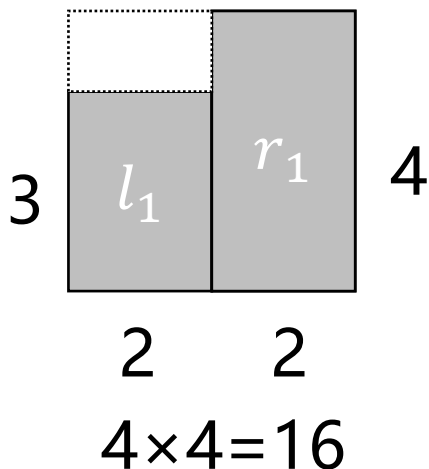
□ 自下而上遍历树

– 访问节点a: **切割方式是垂直的, 调用垂直切割的合并**



– $L = \{(2,3), (3,2)\}$, $R = \{(2,4), (4,2)\}$

1. 合并 $l_1 = (2,3)$ 和 $r_1 = (2,4)$, 得到 $(2 + 2, \max\{3,4\}) = (4,4)$, 由于列表L是按照**高度降序排序**的, l_1 的高度已经小于 r_1 , l_2 的高度比 l_1 更低, **空白部分更多如下所示, 不可能是更优解, 所以下一步应该合并 l_1 和 r_2**



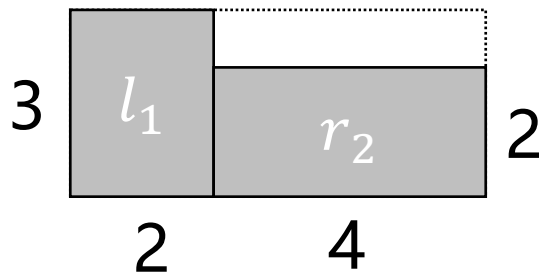
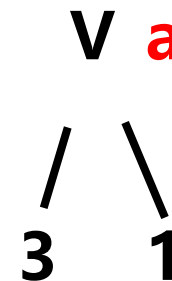
■ Stockmeyer算法

□ 自下而上遍历树

– 访问节点a：切割方式是垂直的，调用垂直切割的合并

– $L = \{(2,3), (3,2)\}$, $R = \{(2,4), (4,2)\}$

2. 合并 $l_1 = (2,3)$ 和 $r_2 = (4,2)$ ，得到 $(2 + 4, \max\{3,2\}) = (6,3)$ ，由于列表R是按照高度降序排序的， r_2 的高度已经小于 l_1 。如果 r_3 存在，那么它的高度比 r_2 更低，空白部分更多，不可能是更优解，所以下一步应该合并 l_2 和 r_2



由于 r_2 没有后续项，所以没有画出图示 6×3

■ Stockmeyer算法

□ 自下而上遍历树

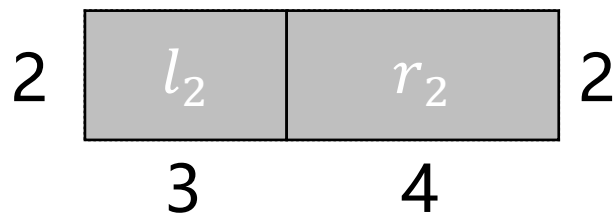
– 访问节点a：切割方式是垂直的，调用垂直切割的合并

– $L = \{(2,3), (3,2)\}$, $R = \{(2,4), (4,2)\}$

3. 合并 $l_2 = (3,2)$ 和 $r_2 = (4,2)$ ，得到 $(3 + 4, \max\{2,2\}) = (7,2)$ 。由于 l_2 和 r_2 的高度相同，同理，合并 l_3 和 r_2 与合并 l_2 和 r_3 不可能是更优解，所以下一步应该合并 l_3 和 r_3 。

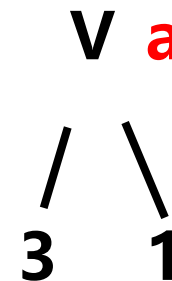
由于 l_2 没有后续项 l_3 和 r_2 没有后续项 r_3 ，合并完成，得到a节点的列表

$D = \{(4,4), (6,3), (7,2)\}$



因为 l_2 和 r_2 没有后续项，所以没有画出图示

7×2



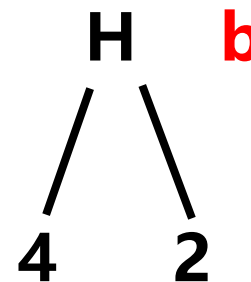
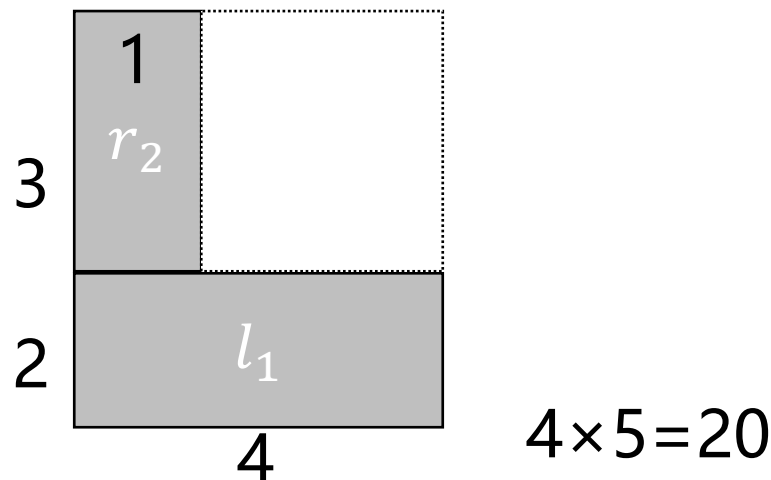
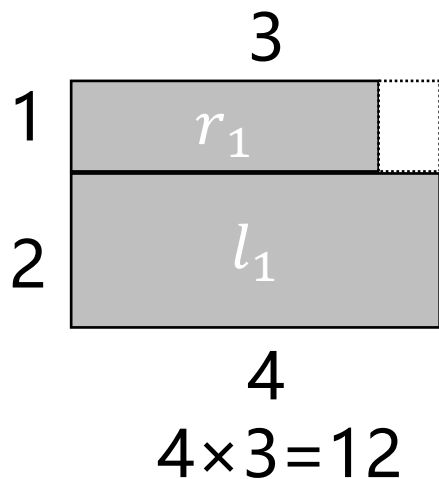
■ Stockmeyer算法

□ 自下而上遍历树

– 访问节点b: **切割方式是水平的, 调用水平切割的合并**

– $L = \{(\overset{l_1}{4}, \overset{l_2}{2}), (\overset{r_1}{3}, \overset{r_2}{1})\}$, $R = \{(3,1), (1,3)\}$

1. 合并 $l_1 = (4,2)$ 和 $r_1 = (3,1)$, 得到 $(\max\{4,3\}, 2 + 1) = (4,3)$, 由于列表R是按照**宽度降序排列**的, r_1 的宽度已经小于 l_1 , r_2 的宽度比 r_1 更低, **空白部分更多如下所示**, 不可能是更优解, 所以下一步应该合并 l_2 和 r_1



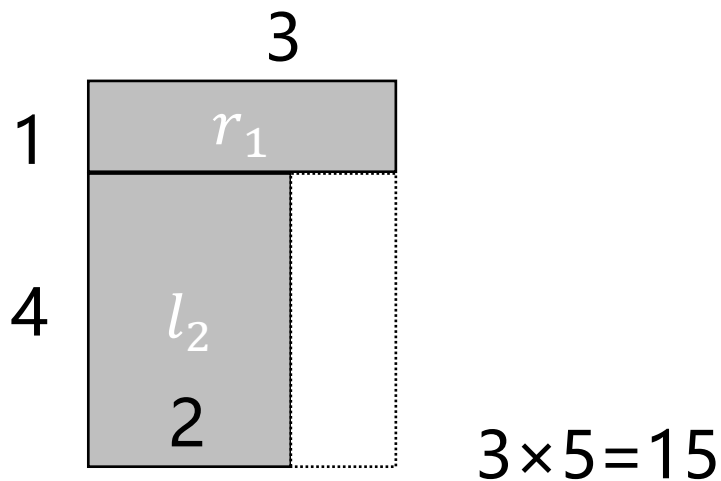
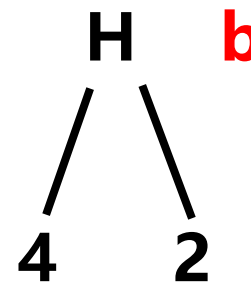
■ Stockmeyer算法

□ 自下而上遍历树

– 访问节点b: 切割方式是水平的, 调用水平切割的合并

– $L = \{(4,2), (2,4)\}$, $R = \{(3,1), (1,3)\}$

2. 合并 $l_2 = (2,4)$ 和 $r_1 = (3,1)$, 得到 $(\max\{2,3\}, 4 + 1) = (3,5)$, 由于列表L是按照宽度降序排列的, l_2 的宽度已经小于 r_1 , l_3 的宽度比 l_2 更低, 空白部分更多, 肯定不是更优解, 所以下一步应该合并 l_2 和 r_2



因为 l_2 没有后续项, 所以没有画出图示

■ Stockmeyer算法

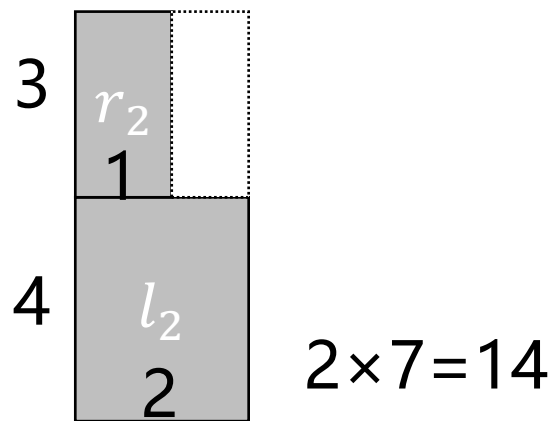
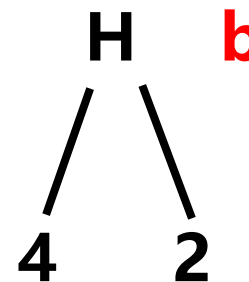
□ 自下而上遍历树

– 访问节点b：切割方式是水平的，调用水平切割的合并

– $L = \{(4,2), (2,4)\}$, $R = \{(3,1), (1,3)\}$

3. 合并 $l_2 = (2,4)$ 和 $r_2 = (1,3)$, 得到 $(\max\{2,1\}, 4 + 3) = (2,7)$, 由于列表R是按照宽度降序排列的, r_2 的宽度已经小于 l_2 , r_3 的宽度比 r_2 更低, 空白部分更多, 一定不是更优解, 所以下一步应该合并 l_3 和 r_2 。

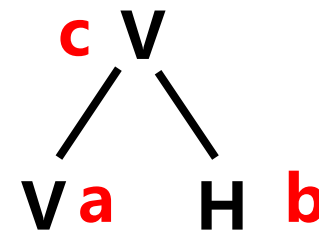
由于 l_2 没有后续项 l_3 , 合并完成, 得到b节点的列表为 $D = \{(4, 3), (3, 5), (2, 7)\}$



因为 l_2 没有后续项, 所以没有画出图示

■ Stockmeyer算法

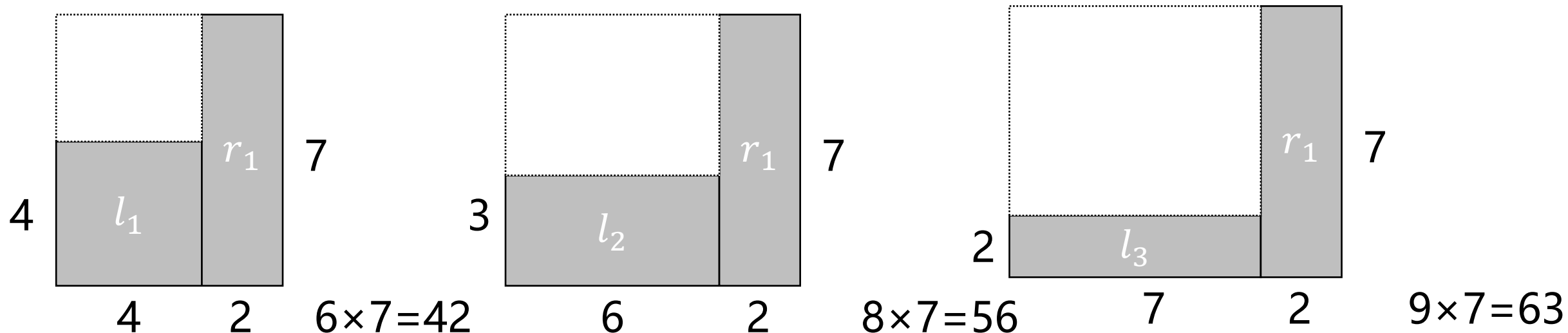
□ 自下而上遍历树



– 访问节点c: 切割方式是垂直的, 调用垂直切割的合并

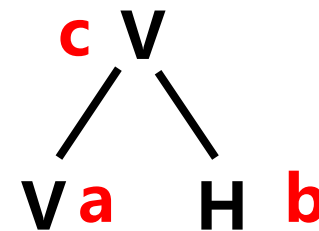
– $L = \{(4,4), (6,3), (7,2)\}$, $R = \{(2,7), (3,5), (4,3)\}$

1. 合并 $l_1 = (4,4)$ 和 $r_1 = (2,7)$, 得到 $(4 + 2, \max\{4,7\}) = (6,7)$, 由于列表L是按照高度降序排列的, l_1 的高度已经小于 r_1 , l_2 和 l_3 的高度比 l_1 更低, 空白部分更多如下所示, 一定不可能是更优解, 所以下一步应该合并 l_1 和 r_2



■ Stockmeyer算法

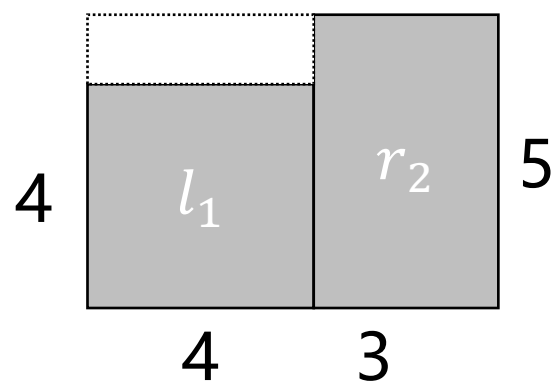
□ 自下而上遍历树



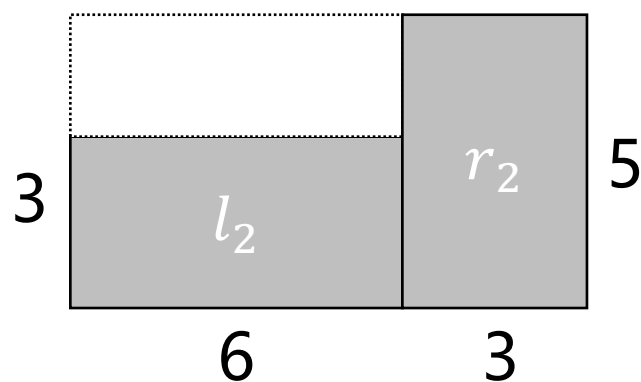
– 访问节点c: 切割方式是垂直的, 调用垂直切割的合并

– $L = \{(4,4), (6,3), (7,2)\}$, $R = \{(2,7), (3,5), (4,3)\}$

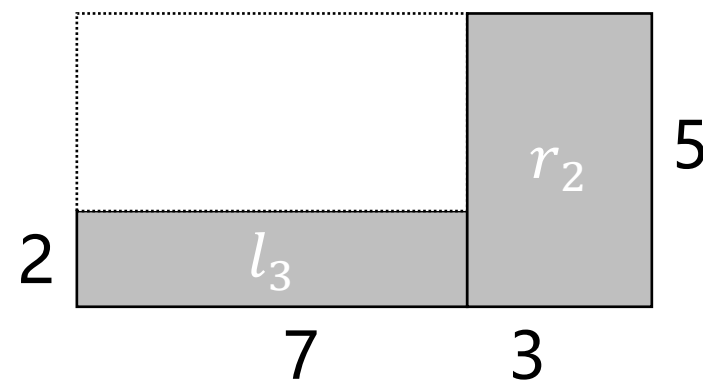
2. 合并 $l_1 = (4,4)$ 和 $r_2 = (3,5)$, 得到 $(4 + 3, \max\{4,5\}) = (7,5)$, 由于列表L是按照高度降序排列的, l_1 的高度已经小于 r_2 , l_2 和 l_3 的高度比 l_1 更低, 空白部分更多 (如下所示), 所以下一步应该合并 l_1 和 r_3



$$7 \times 5 = 35$$



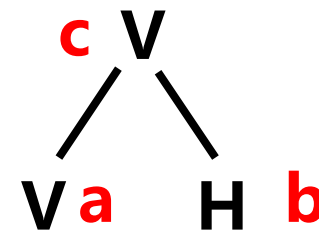
$$9 \times 5 = 45$$



$$10 \times 5 = 50$$

■ Stockmeyer算法

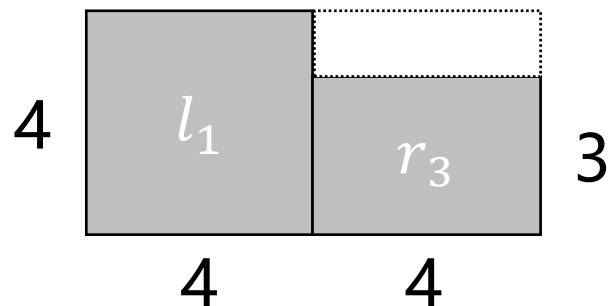
□ 自下而上遍历树



– 访问节点c: 切割方式是垂直的, 调用垂直切割的合并

– $L = \{(4,4), (6,3), (7,2)\}$, $R = \{(2,7), (3,5), (4,3)\}$

3. 合并 $l_1 = (4,4)$ 和 $r_3 = (4,3)$, 得到 $(4 + 4, \max\{4,3\}) = (8,4)$, 由于列表R是按照高度降序排列的, r_3 的高度已经小于 l_1 , r_4 (如果存在的话) 的高度比 r_3 更低, 空白部分更多, 一定不是更优解, 所以下一步应该合并 l_2 和 r_3

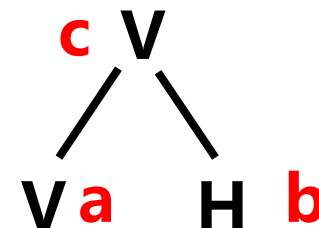


$$8 \times 4 = 32$$

因为 r_3 没有后续项, 所以没有画出图示

■ Stockmeyer算法

□ 自下而上遍历树



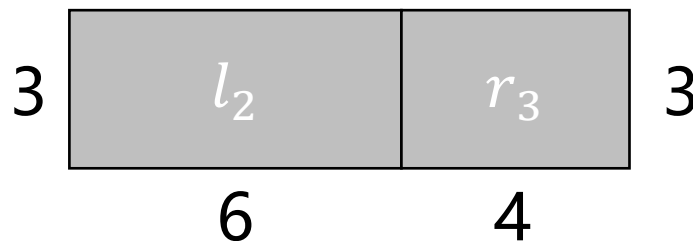
– 访问节点c: 切割方式是垂直的, 调用垂直切割的合并

– $L = \{(4,4), (6,3), (7,2)\}$, $R = \{(2,7), (3,5), (4,3)\}$

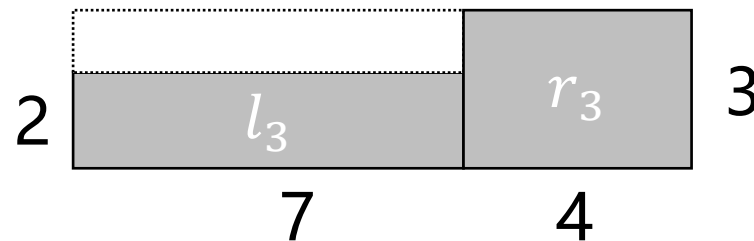
4. 合并 $l_2 = (6,3)$ 和 $r_3 = (4,3)$, 得到 $(6 + 4, \max\{3,3\}) = (10,3)$, 由于 l_2 和 r_3 的高度相同, 同理, 合并 l_3 和 r_3 与合并 l_2 和 r_4 (如果存在的话) 一定不可能是更优解 (如下所示), 下一步应该合并 l_3 和 r_4 。

由于 r_3 没有后续项 r_4 , 所以合并完成。得到c节点的列表 $D = \{(6, 7), (7, 5), (8, 4), (10, 3)\}$

c是根节点, 根节点的最小面积为 $10 \times 3 = 30$



$$10 \times 3 = 30$$

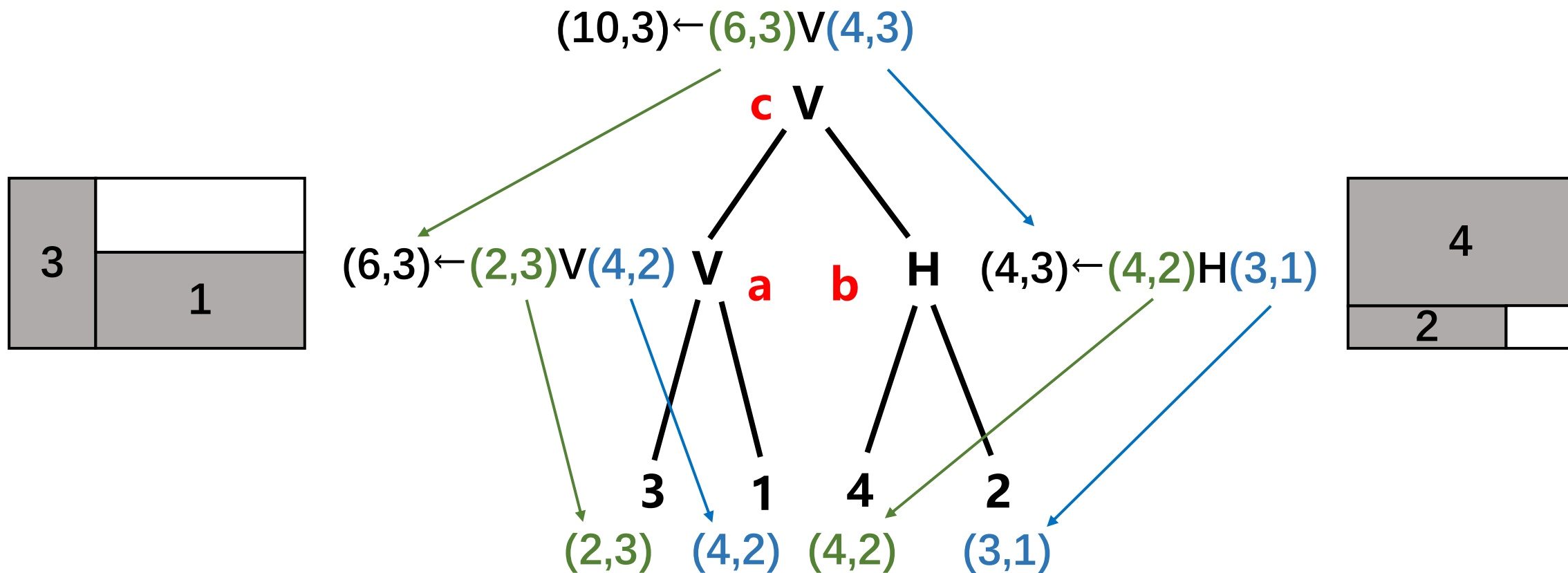


$$11 \times 3 = 33$$

■ Stockmeyer算法

□ 自上而下遍历树

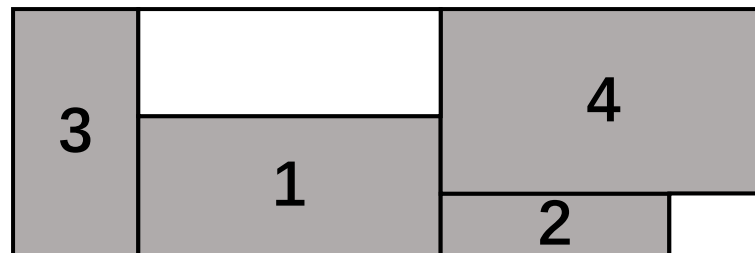
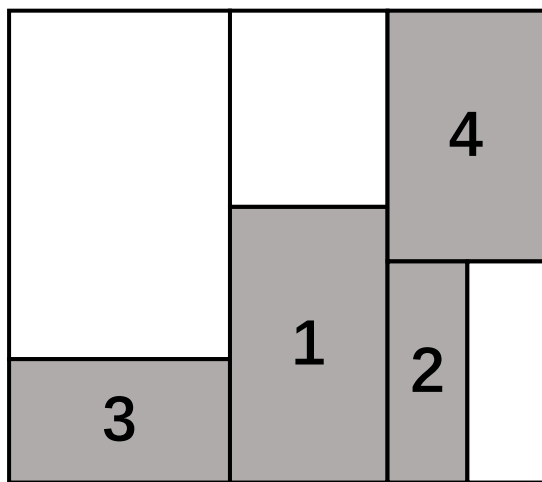
- 最后自上而下遍历树，确定每个模块的放置方式。



■ Stockmeyer算法

□ 最终结果

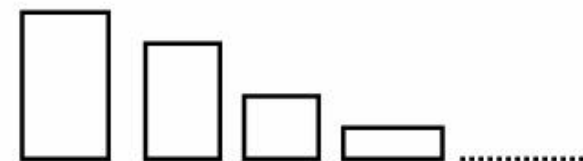
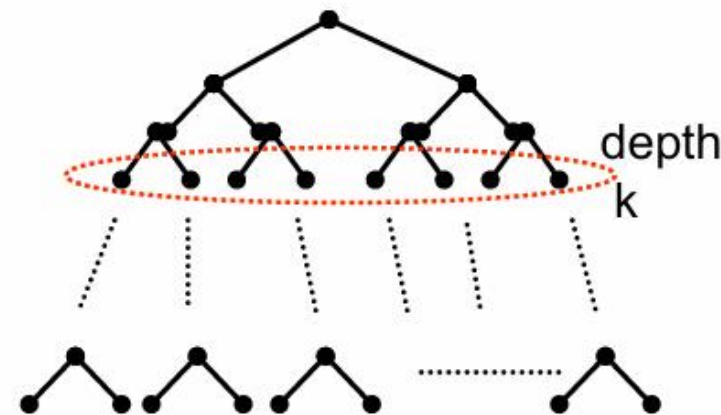
– 面积从 7×7 变为 10×3 (维持初始切割方式)



■ Stockmeyer算法

□ 时间复杂度分析

- 假设叶子节点数量为 n ，树的深度为 d
- 时间复杂度 $O(nd)$ ，空间复杂度 $O(nd)$
- 因为在深度 k 处
 - 列表长度之和 $O(n)$
 - 构建这些列表的时间 $O(n)$ ，由前面的算法易得
 - 每一层都需要保留合并来源的信息
- 如果每个模块有 m 种可能的形状
- 时间复杂度和空间复杂度都是 $O(nmd)$



■ Stockmeyer算法

□ 总结

- 对于**给定的模块集合**，Stockmeyer算法可以提供最优解
 - 但这是一个非常理想的情况，不常发生
- 缺点
 - **无法处理非切片式布图规划**
 - **输出结果受限于初始的切片树，对初始解敏感**

PART 03

搜索算法

■ 搜索算法

□ 问题定义

– 给定 n 个模块 $S = \{1, 2, \dots, n\}$, 每个模块的约束三元组 (A_1, r_1, s_1) 要求满足

- $A_i = w_i h_i$, $r_i \leq h_i / w_i \leq s_i$, 如果 $i \in$ 软模块
- 模块间不重叠
- 整个布图的面积最小
- 预估的总线长最小

– 输出

- 一个合法的布图 R
- 每个模块的长宽 $w_i h_i$
- 每个模块的位置

■ 搜索算法

□ 搜索算法

- 核心算法是**模拟退火**
- 不同之处：在于对布图规划问题的**建模方式**
- 建模思路的核心是：把**二维平面的相对位置关系**压缩到**一维的先后关系**

■ 切片树

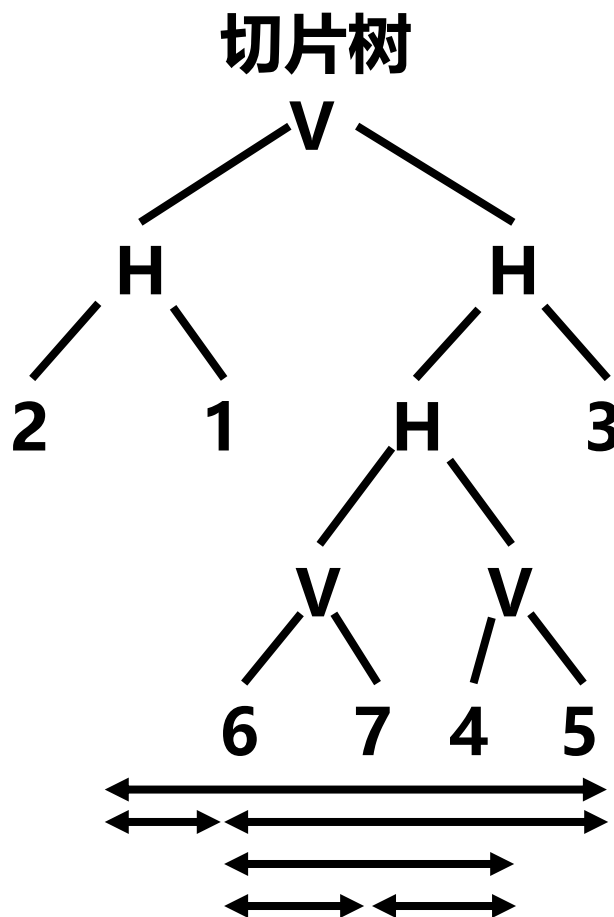
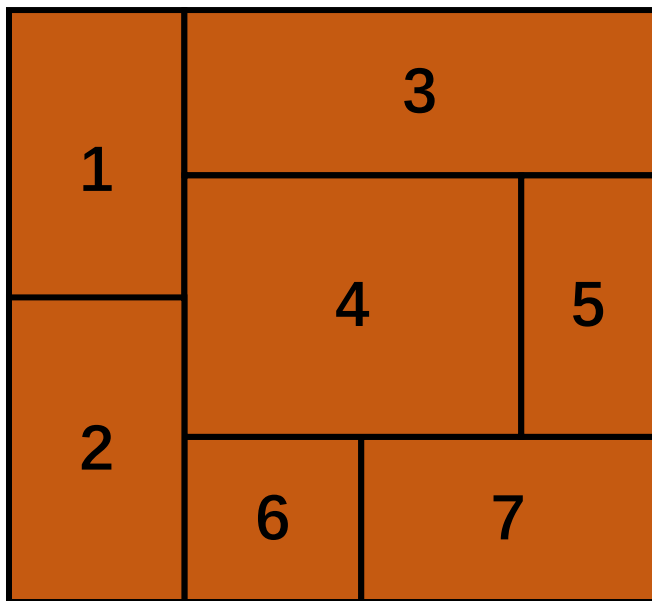
□ 问题建模

– 波兰表达式

- 切片树表示切片式布图规划，波兰表达式由切片树的**后序遍历**生成。
- 将**二维的平面规划问题**转化为**一维的字符串排列优化问题**。

➤ 切片式布图规划的表示

切片式布图规划



波兰表达式
(切片树的后序遍历)

21H67V45VH3HV

H表示水平切，V表示垂直切

■ 切片树

□ 波兰表达式

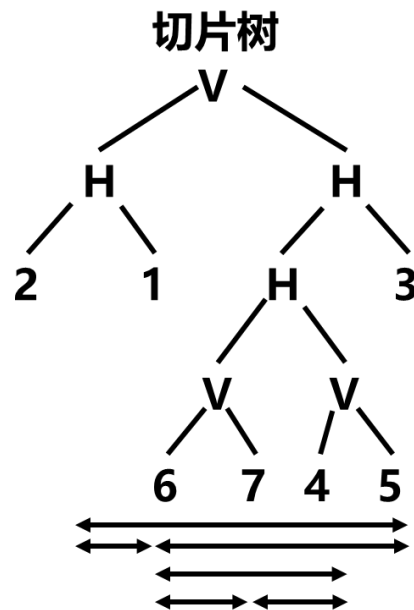
– 切片式布图规划的简洁表示

➤ 大致指定模块的相对位置

– 切片树的后序遍历

1. 左子树的后序遍历
2. 右子树的后序遍历
3. 当前根节点的标签

– 对于 n 个模块，波兰表达式包含 n 个操作数(模块)和 $n-1$ 个运算符(H, V)



波兰表达式
(切片树的后序遍历)

21H67V45VH3HV

□然而，对于给定的切片式布图规划，对应的切片树(以及波兰表达式)并不唯一。因此，这种表示存在一定的冗余

■ 切片树

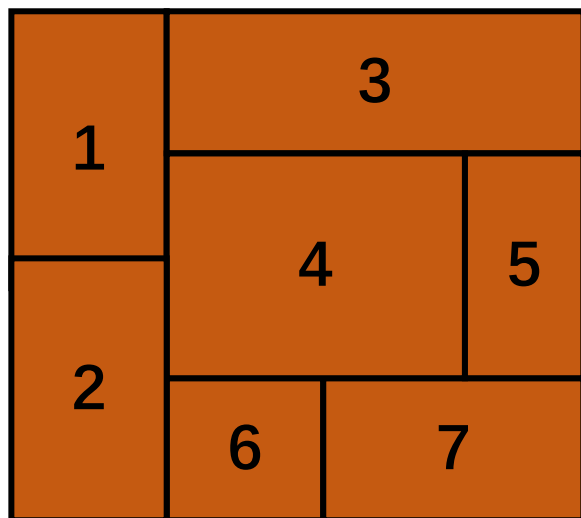
□ 定义：**偏斜切片树**(Skewed Slicing Tree)

– 要求：没有一个节点和它的右子节点是相同的

□ **规范化波兰表达式**(Normalized Polish Expression)

– 表达式中没有连续的H或V

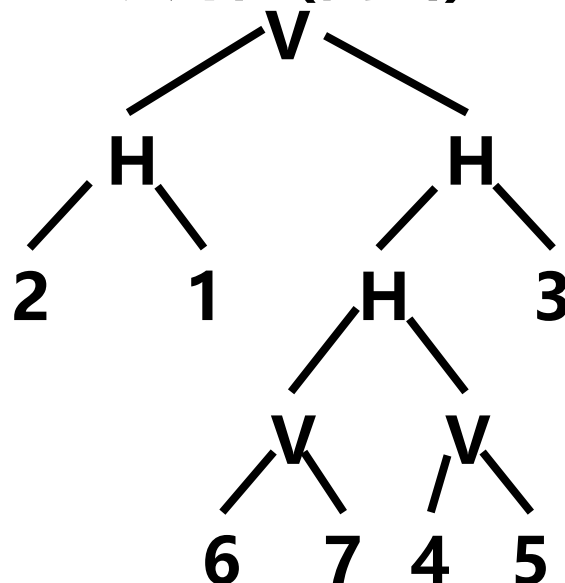
切片式布图规划



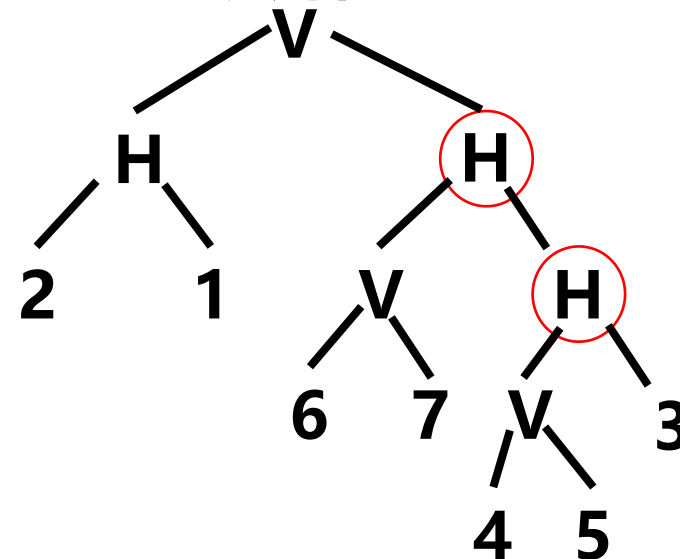
波兰表达式

21H67V45VH3HV

切片树(偏斜)



切片树



21H67V45V3HHV

■ 切片树

□ 规范化波兰表达式

- 切片式布图规划、偏斜切片树和规范化波兰表达式之间存在一一对应的关系
- 使用规范化波兰表达式来表示切片式布局规划
- 这样布图规划的二维问题表述为一维字符串的状态空间搜索问题

■ 切片树

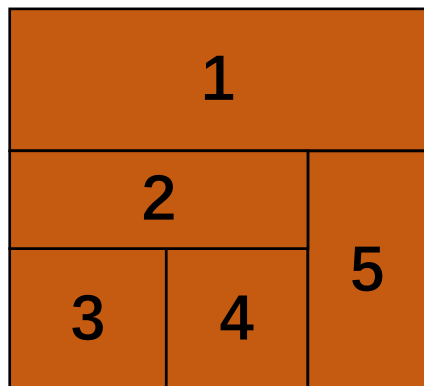
□ 邻域结构(Neighborhood Structure)

- 链 (Chain) : HVHVH... 或 VHVHV...



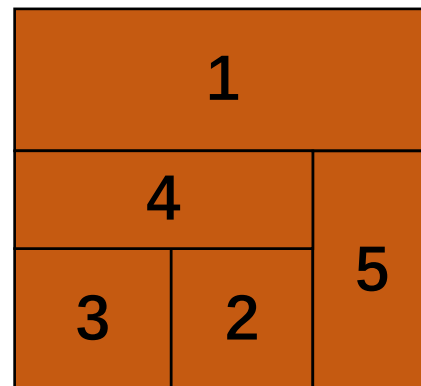
- 移动操作包括:
 - M1: 交换相邻的操作数 (忽略链)
 - M2: 对某个链取反 (H变V, V变H)
 - M3: 交换两个相邻的操作数和运算符(注意: M3 可能会生成无效的规范化波兰表达式, 因此需要在 M3 后检查有效性)
- 可以证明, **每对有效的规范化波兰表达式都是相互连通的**

■ 切片树



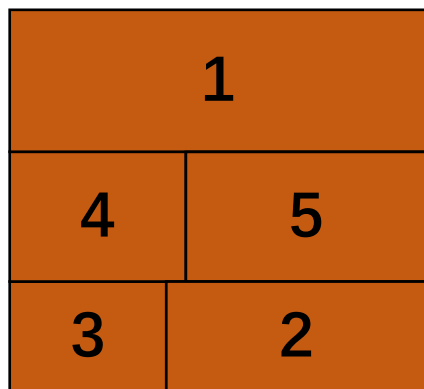
34V2H5V1H

M1
交换2和4的位置



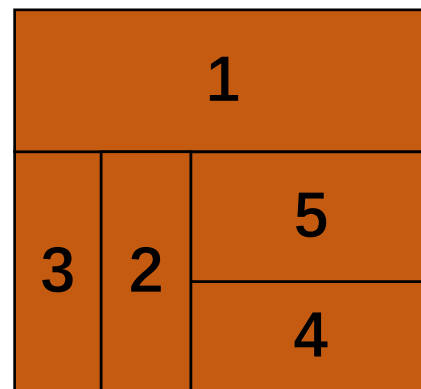
32V4H5V1H

交换H和5的位置
M3



32V45VH1H

M2
HV取反



32V45HV1H

■ 切片树

这样做存在问题？

只解决布图规划中模块放置问题，即确定模块间的**相对位置**关系，
但无法确定模块的形状。

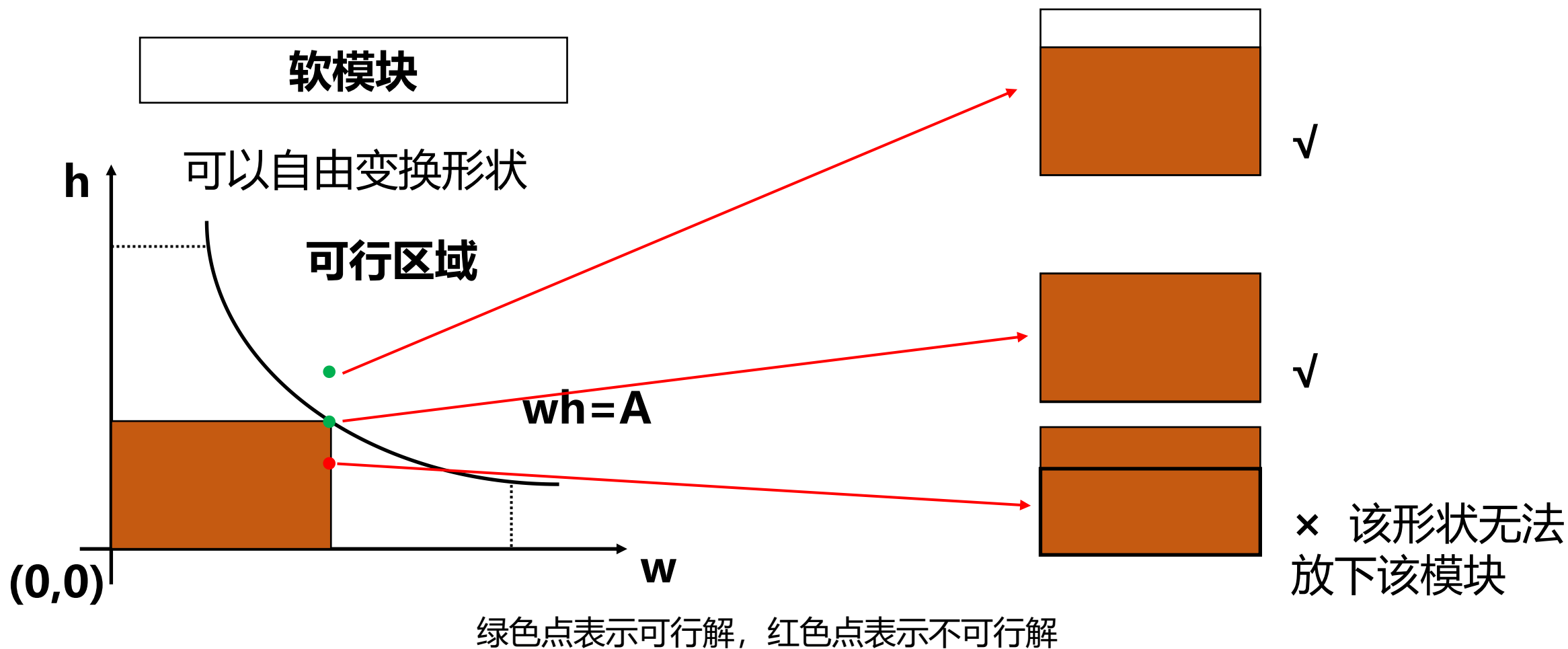
如何解决？

引入可行形状曲线来解决这个问题。

■ 切片树

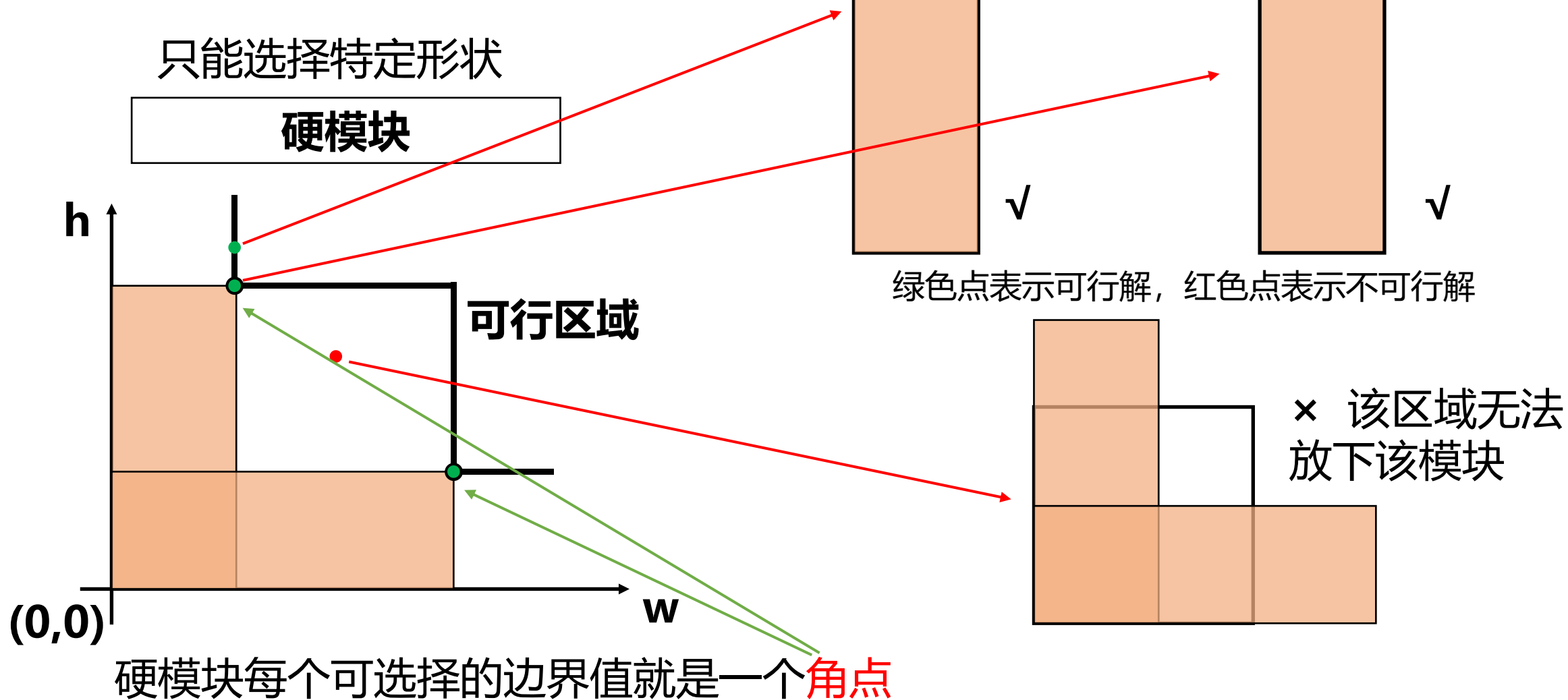
□ 形状曲线(shape curve)

– 表示模块的可行形状



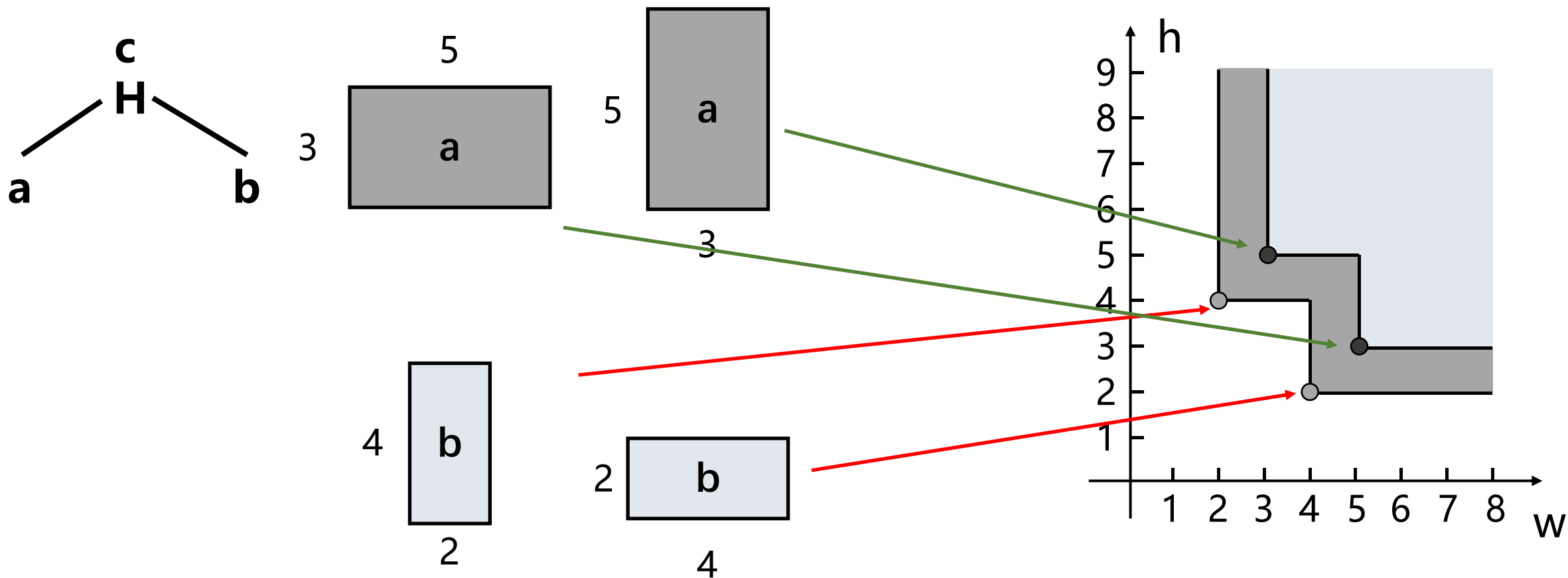
■ 切片树

□ 形状曲线



■ 切片树

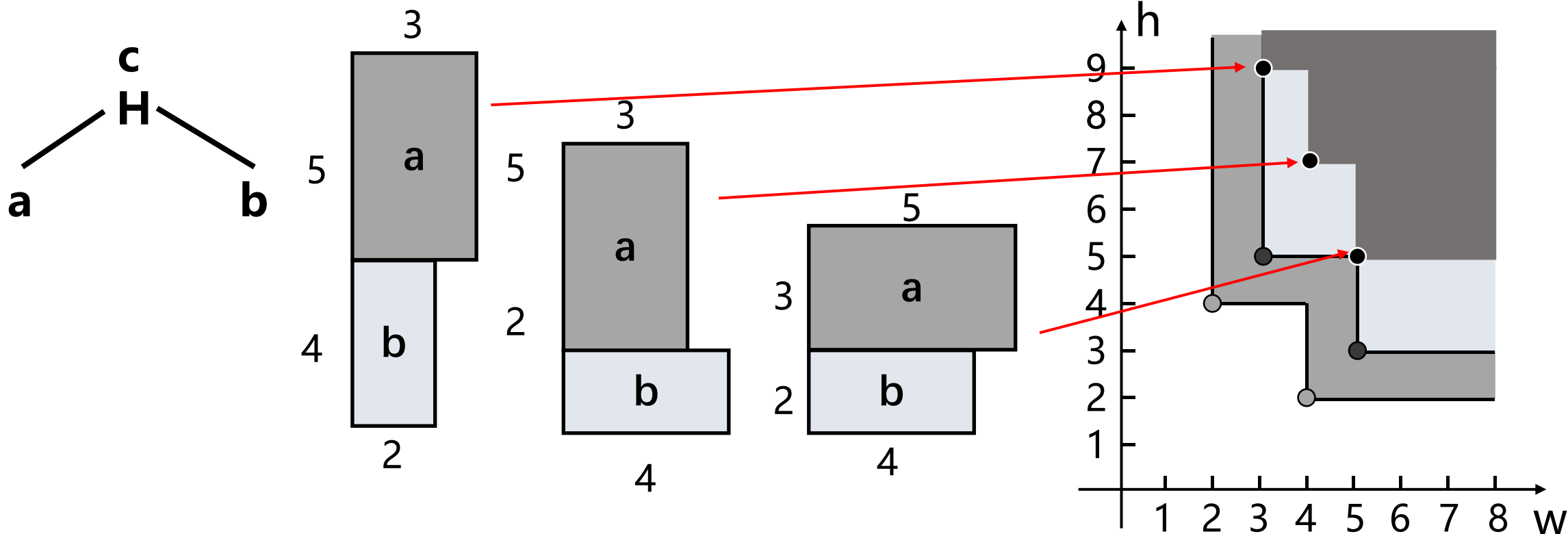
□ 给定两个硬模块a, b, 先构建各自的形状曲线



■ 切片树

□ 叠加得到整个组合的形状曲线

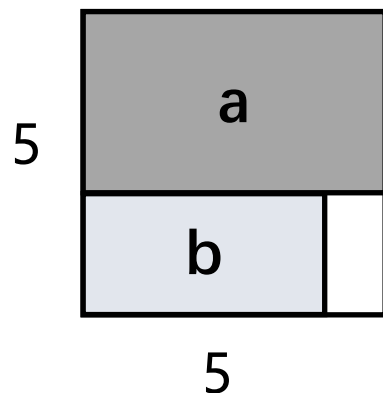
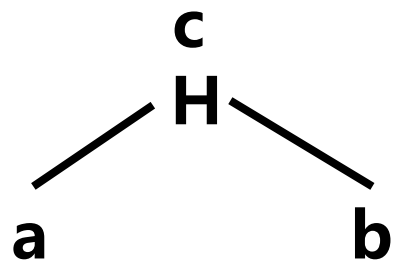
叠加找出所有角点，构建整个模块组合的形状曲线，宽度取子模块的最大值。**同时记录角点的来源**



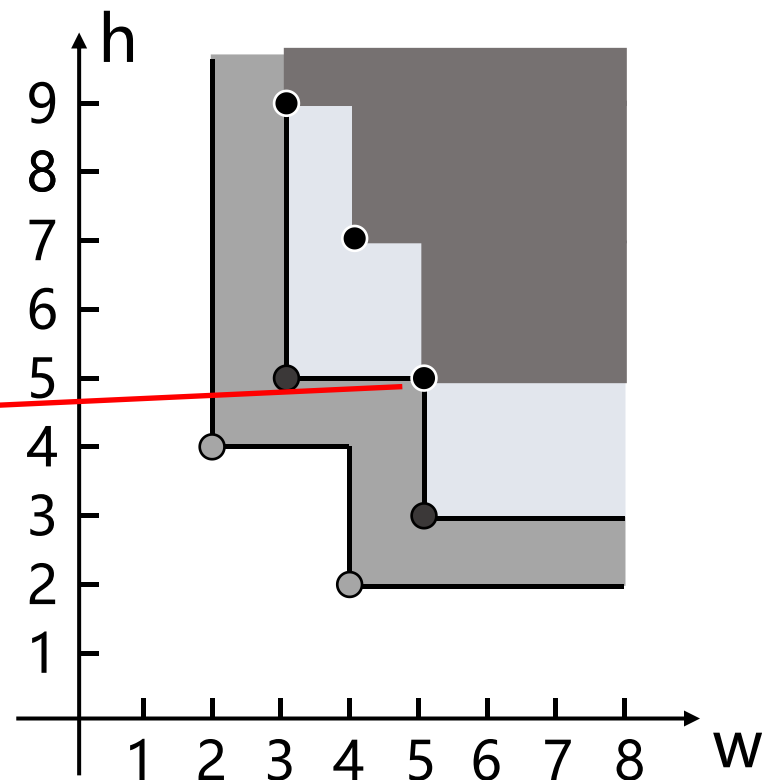
■ 切片树

□ 在根节点c处确定最小面积

- 根据角点记录的来源，确定模块的形状



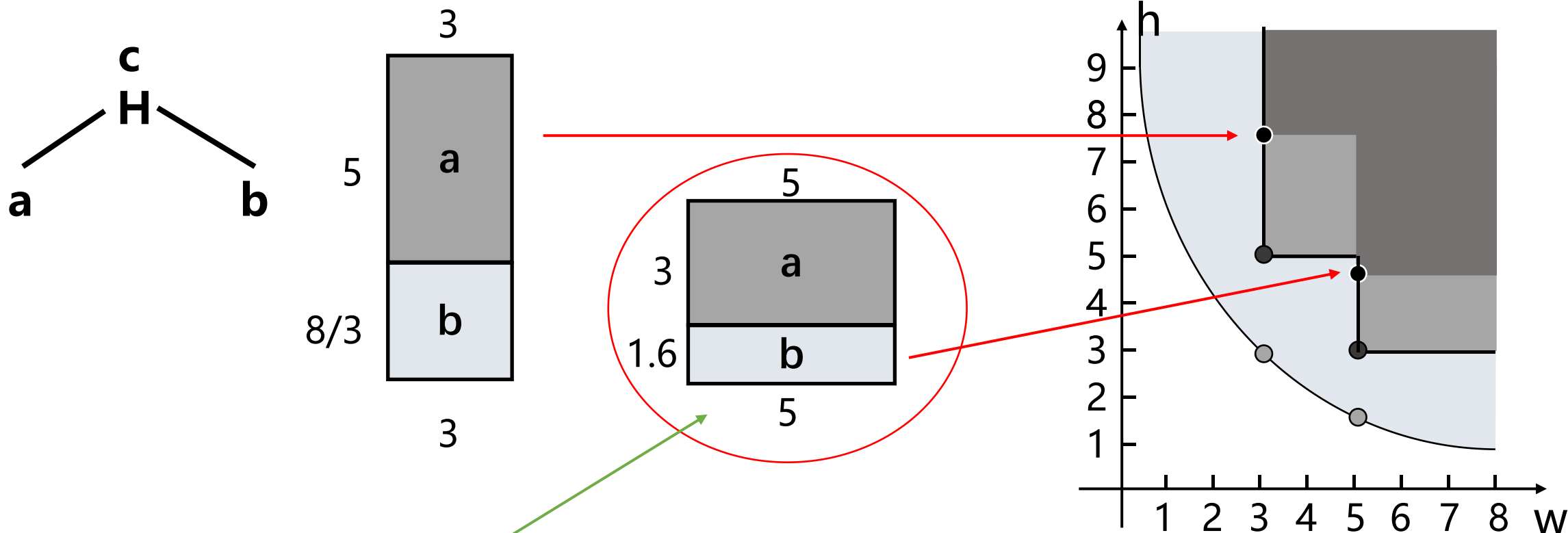
最小面积为 $5 \times 5 = 25$
确定模块ab的摆放



■ 切片树

□ 如果b是软模块

- 那就改变b的形状保证布局没有空白区域，得到角点

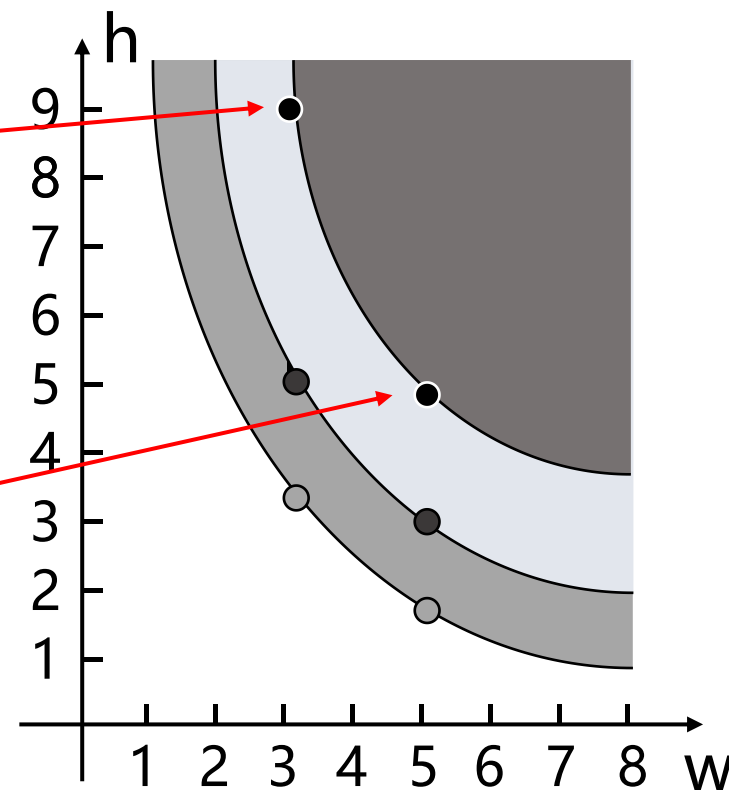
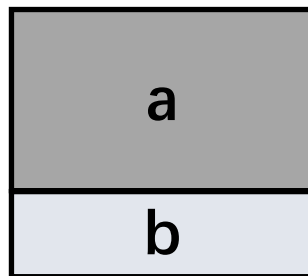
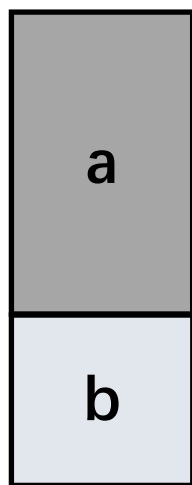
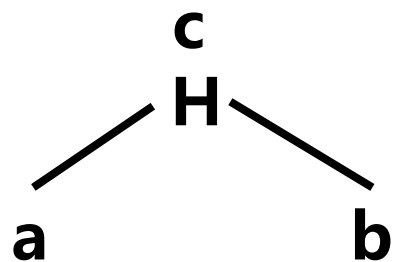


选择面积最小的角点，确定模块的摆放

■ 切片树

□ 如果都是软模块

- 那整条曲线都是当前组合的角点，在下一层处理时把c当作是一个软模块



形状曲线上表示的形状面积均相同

■ 切片树

□ 布图规划设计更新后调整模块形状

- 如果为每个模块形状保留 k 个点，则每个规范化波兰表达式的形状曲线计算时间复杂度为 $O(kn)$
- 每次操作后，布图规划的变化较小，因此不需要从头开始重新计算模块形状
- 我们可以在每次操作后逐步更新模块形状
- 时间复杂度约为 $O(k \log n)$

■ 切片树

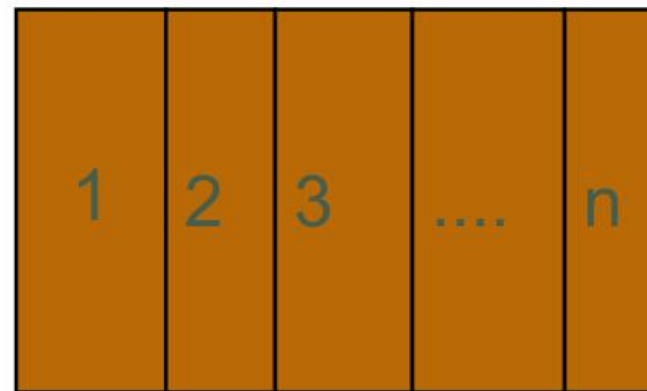
□ 初始解

– 前面解决模块相对空间位置和模块形状的问题

- 解空间定义
- 状态定义
- 相邻状态定义
- 模块形状定义

– 最后定义搜索算法的初始解

- $12V3V4V...nV$



■ 切片树

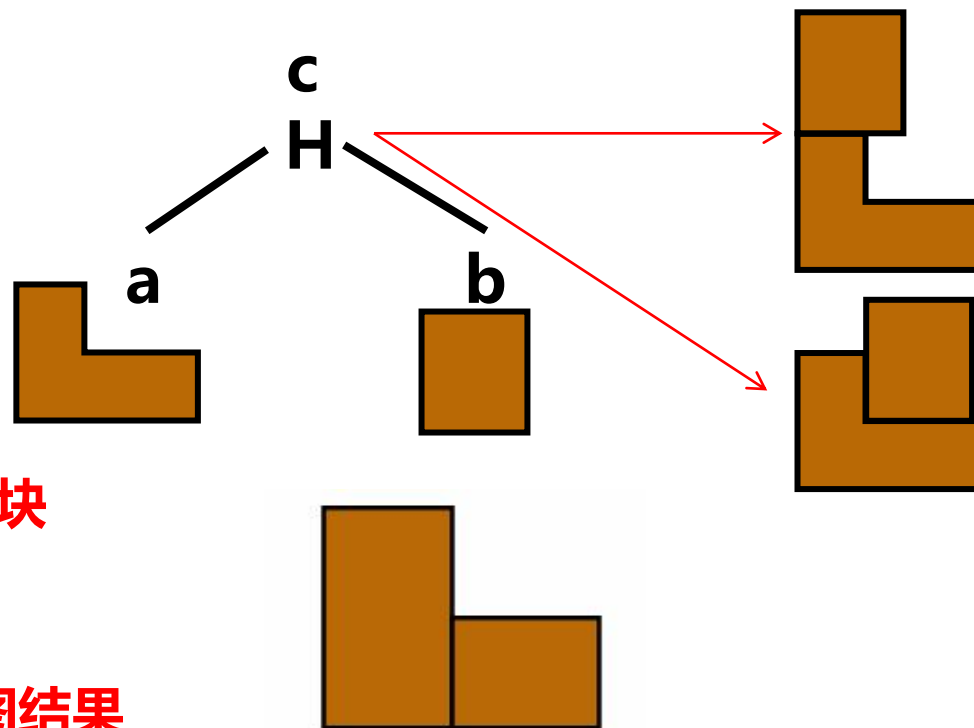
□ 降温策略

- $T_i = \alpha T_{i-1}$, 其中 $\alpha = 0.85$
- 在每个温度下, 尝试 $k \times n$ 次操作(k 在5到10之间, n 是超参数)
- 如果满足以下任一条件, 则终止退火过程
 - 接受移动的总数占全部移动比例 < 5%
 - 温度足够低

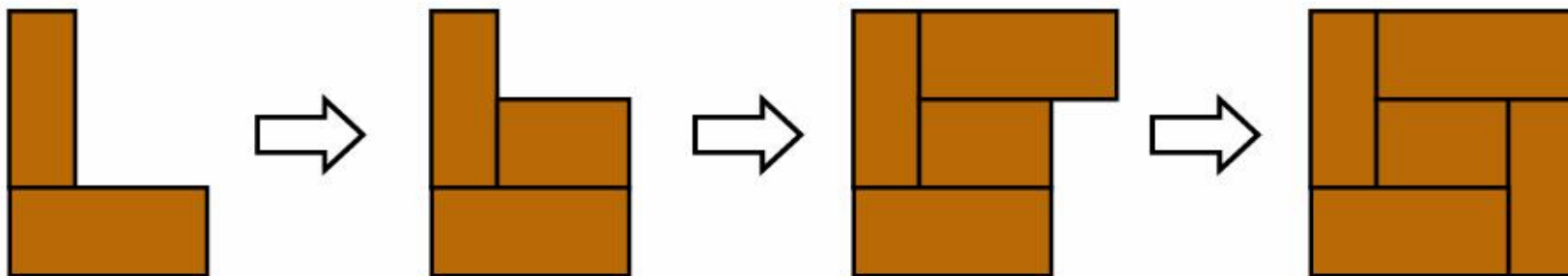
■ 切片树

□ 前面提到的建模方法**不能**解决以下的问题

- **L形模块**，假设右边切片树中 a 是L型模块
 - 显然右边两种组合方式都符合 abH 的切片树
 - 一个 H 表示的组合方式存在歧义，高度会不同
- **即使初始状态只有矩形模块，也可能生成L形模块**



- **另外：只使用矩形模块也可能生成非切片式布图结果**



■ 切片树

□ 为了解决上述非切片式布局和L型模块的问题

- Wong & Liu 在87年^[1]做出改进
- 与DAC-86的论文^[2]类似：
 - 使用波兰表达式表示
 - 通过简单的操作局部修改布图规划(floorplan)
 - 模拟退火
 - 仍然属于切片式布局
- 与DAC-86论文的不同之处：
 - **定义5种运算符，能处理L型模块的问题**
 - 不再使用形状曲线(shape curve)的概念
 - 采用模拟退火算法以概率方式为模块选择不同的形状

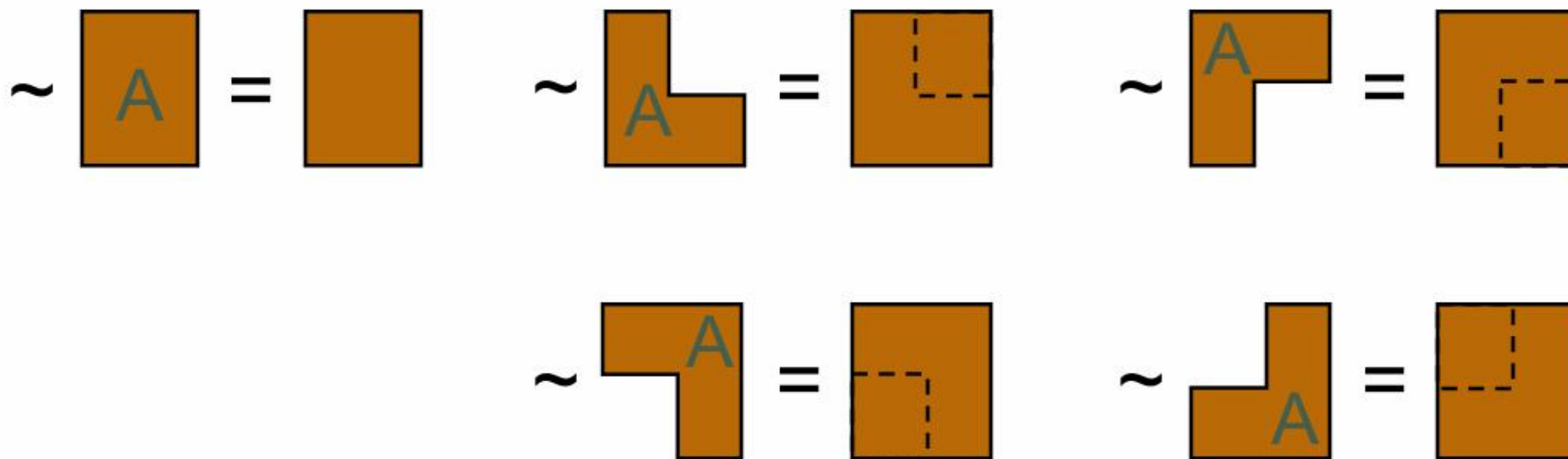
[1] D.F. Wong and C.L. Liu, "Floorplan Design for Rectangular and L-shaped Modules," ICCAD, 1987.

[2] D.F. Wong and C.L. Liu, "A New Algorithm for Floorplan Design," DAC, 1986.

■ 切片树

□ 5种波兰表达式运算符: $\sim$, V_1 , V_2 , H_1 , H_2

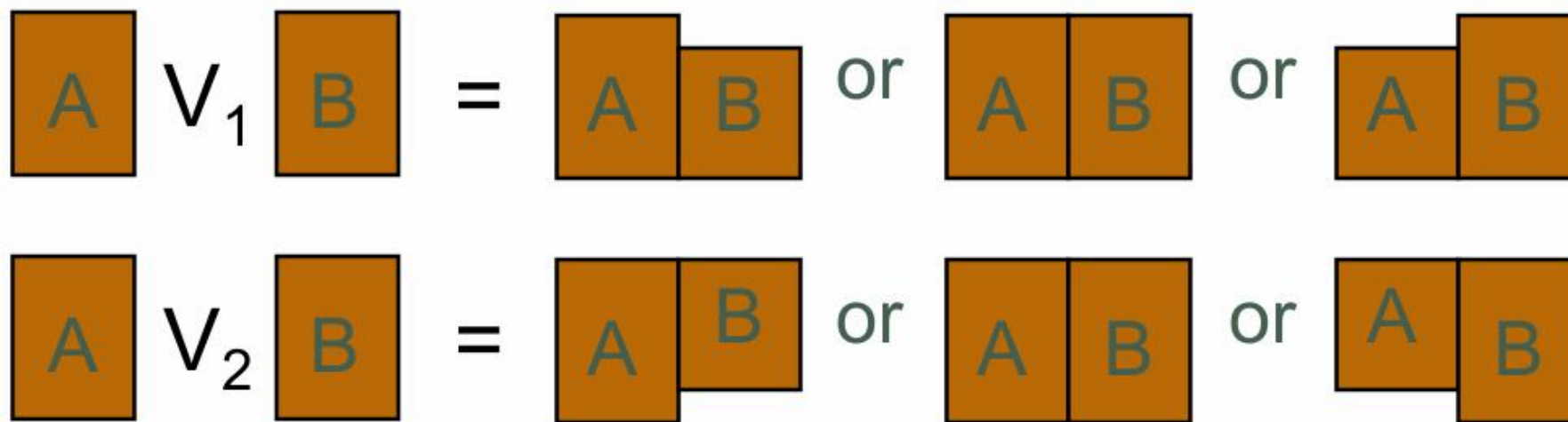
□ 补运算($\sim A$): 对L型模块缺少部分补足, 使得整体再次成为矩形



■ 切片树

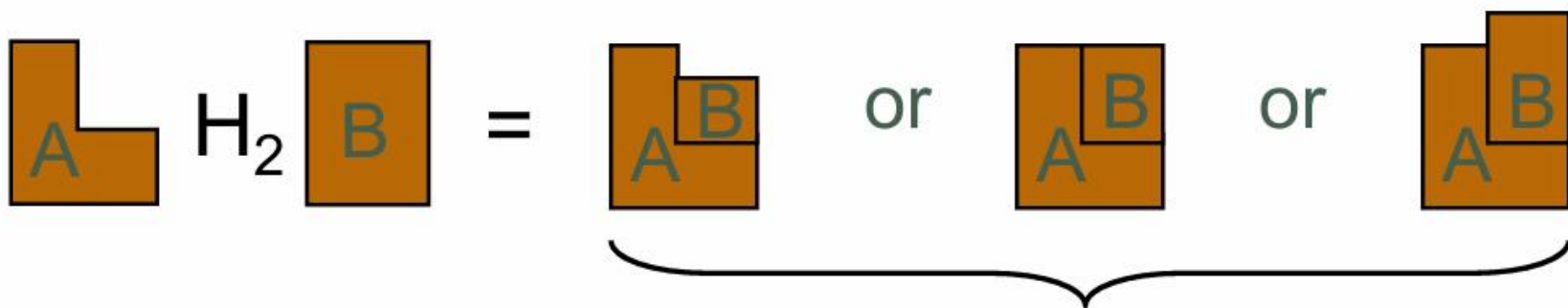
□ V_1 和 V_2 组合的例子

– V_1 是底部对齐, V_2 是顶部对齐



■ 切片树

□ 组合的其他例子



表示为



■ 切片树

□ 新的邻域移动方式

- 将波兰表达式写成以下形式：

$$b_1u_1b_2u_2\cdots b_{2n-1}u_{2n-1}$$

- 其中 b_i 代表 n 个模块或 $n-1$ 个二元运算符中的一个，每个 u_i 可以是 $\sim$ 或空字符串 ε

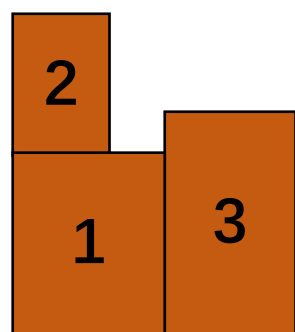
- 注意是后缀表达式，因此 u_i 在修饰的 b_i 后面

- 操作包括：

- M1：对某些 b_i 进行修改（改变成不同的形状或不同的二元运算符）
 - M2：将某个 u_i 改为 $\sim$ 或空
 - M3：交换有相同类型（同为模块或同为操作符） b_i 和 b_j
 - M4：交换相邻的 b_i 和 b_{i+1} （M4操作可能导致无效的波兰表达式，需进行检查）

■ 切片树

□ 例子

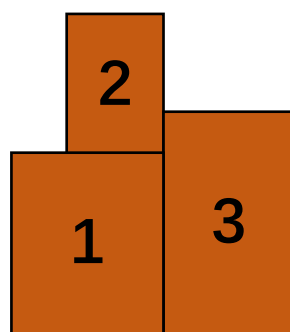


$12H_13V_1$

M1



把 H_1 变为 H_2

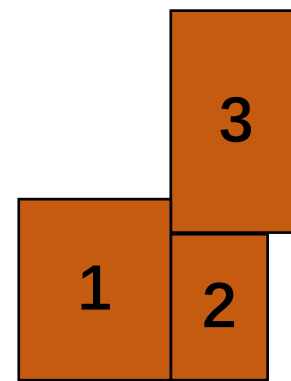


$12H_23V_1$

M3



交换 H_2 和 V_1



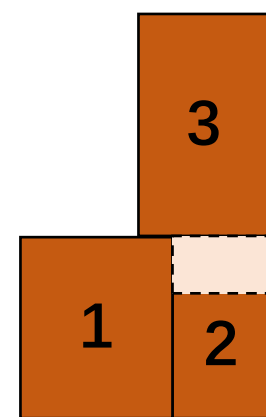
$12V_13H_2$

u_3 位置变成~

M2



普通模块和普通模块按 H_2 (右对齐)组合

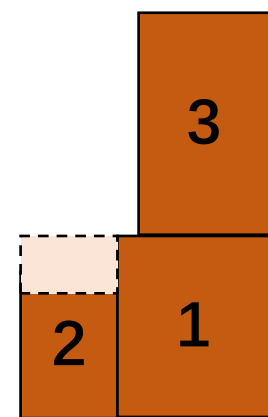


$12V_13H_2$

M4



交换 b_1 和 b_2
(交换1和2)



$21V_13H_2$

■ 切片树

□ 总结

- 使用切片树表示模块间相对位置关系
- 利用规范化波兰表达式来对应唯一切片树
 - 把复杂的二维平面相对位置用一维信息表示，降低解空间表示难度
- 不足之处
 - 解空间没有完全覆盖整个设计空间，不能探索非切片树类型的解，可能导致结果的次优

感谢！
